

Polynomial Regression: Assignment Report

Roll number: BT2024171. Models: polynomial regression (ridge-regularised), built with scikit-learn.

GitHub repository (all training and inference code): <https://github.com/Ayush1patel/ML-Assignment1-Polynomial-Regression>

1. Approach

For each problem the features are expanded to all monomials of total degree $\leq d$ (PolynomialFeatures, no constant column; the intercept is fitted by Ridge). The model is $y = w^T \phi(x) + b$. Parameters minimise

$$\sum_i (y_i - w^T \phi(x_i) - b)^2 + \alpha ||w||^2,$$

i.e. ordinary least-squares polynomial regression plus an L2 (ridge) penalty α (the intercept is not penalised). The only inputs are the polynomial terms of $x_1..x_k$, so this is purely polynomial regression.

2. Why a penalty was needed

With 1000 training samples, a degree- d polynomial has $C(k+d, d) - 1$ terms: 8007 for var1 at $d=10$ and 1770 for var2 at $d=20$, far more than the samples. Even at moderate degree the design matrix is badly conditioned (inputs lie in $[-1, 1]$, so high powers are nearly collinear). Unpenalised least squares gave huge CV errors at high degree (e.g. var2, $d=16$, $\alpha=1e-8$: CV MSE 255). The penalty α is therefore chosen together with the degree.

3. Degree / alpha selection

For every degree d in 1..10 (var1) and 1..20 (var2), α was chosen from a grid by 5-fold cross-validation on the training set (shuffled, fixed seed), and the CV MSE of the best α was recorded. The promising degrees were then re-checked with a finer α grid ($1e-4$.. $1e2$, 13 values) and 3x repeated 5-fold CV. The test files are never used for fitting or selection. Rule: take the lowest degree whose CV MSE is within noise of the minimum (simpler model, safer on the test inputs, see section 5).

Results and Discussion

4. Results

	var1 (6 features)	var2 (3 features)
chosen degree	5	12
ridge alpha	3.162	0.100
number of terms	461	454
train MSE	0.1866	0.1486
train R ²	0.9814	0.9968
5-fold CV MSE	0.4161	0.2351
5-fold CV R ²	0.9584	0.9950

var1: CV MSE falls from 9.05 (d=1) to 0.43 at d=5, then rises again (0.47 at d=6, 0.76 at d=10) as the model overfits (underfitting below 5, overfitting above). d=5 and d=6 are tied in the repeated CV (0.433 vs 0.436); d=5 was chosen as the simpler model. The task statement says the true degree is at most 10; the CV minimum at 5 is consistent with that.

var2: CV MSE drops steeply to about 0.25 by d=8 and then stays flat (0.23-0.27) up to d=20; the true degree is at most 20. Repeated CV with finer alpha gave 0.2375 (d=10), 0.2290 (d=12), 0.2295 (d=14); these are statistically equivalent, so the lowest, d=12, was used. Higher degrees give no gain, and the flat curve suggests the remaining error (about 0.23) is mostly irreducible noise.

5. Other notes

- Distribution shift: the train inputs have std about 0.71 (var1) / 0.68 (var2), but the test inputs are more spread out (std about 0.82 for var1) with more points on the clipped boundary at ± 1 . The model therefore partly extrapolates, where high-degree polynomials can blow up; this is a further reason for ridge and for the lowest adequate degree. Predicted test ranges (var1 -10.5..15.1, var2 -29.5..37.6) are close to the training target ranges (-10.6..14.8, -28.3..40.0), so no explosion was seen. CV on the training data cannot measure this shift, so the true test error may be somewhat higher than the CV figures.
- Data had no missing values; the files were used as given (no extra preprocessing, no feature scaling).
- Metrics: $MSE = \text{mean}(y - \hat{y})^2$; $R^2 = 1 - \text{sum}(y - \hat{y})^2 / \text{sum}(y - \bar{y})^2$.

6. Files (in the repository)

codes/train_predict.py (final training + prediction), codes/cv_explore.py and codes/cv_refine.py (degree/alpha selection), codes/make_report.py (this report), predictions/BT2024171_pred_var1.csv and predictions/BT2024171_pred_var2.csv (one column y, 1000 rows, same order as the test files).

